CIS352 Intro to Enterprise Computing

Section 3 Assignment

- Chapter 13 How to work with sequential files
- The following two chapters will be added if time allows:
  - Chapter 14 How to work with indexed files
  - Chapter 16 How to use the sort/merge feature

Programming Assignment: Sequential File Maintenance (Employee Records)

Objective: Design and write a COBOL program that updates a sequential Employee Master file using a sequential Personnel Transaction file. The program will produce a New Employee Master file and an Error Transaction file for any invalid HR requests.

1. Input and Output Specifications

Input File 1: Old Employee Master File (`OLDEMP`) This file contains the current roster of active employees. The records are 57 bytes long and are sorted in ascending order by Employee ID.

- **Employee ID:** 5 characters (Alphanumeric).
- **Employee Name:** 30 characters (Alphanumeric).
- **Department Code:** 5 characters (Alphanumeric).
- **Job Classification:** 2 characters (Alphanumeric).
- **Annual Salary:** 7 digits (Numeric, `S9(5)V99`).
- **Accrued Vacation Hours:** 3 digits (Numeric, `S9(3)`).
- **Accrued Sick Hours:** 5 digits (Numeric, `S9(3)V99`).

Input File 2: Personnel Transaction File (`EMPTRAN`) This file contains HR requests to add new hires, terminate employees, or update employee details (like a raise or department transfer). The records are 50 bytes long and are sorted in ascending order by Employee ID.

- **Action Code:** 1 character. `A` = Add (Hire), `D` = Delete (Terminate), `C` = Change (Update).
- **Employee ID:** 5 characters (Alphanumeric).
- **Employee Name:** 30 characters (Alphanumeric).
- **Department Code:** 5 characters (Alphanumeric).
- **Job Classification:** 2 characters (Alphanumeric).
- **Annual Salary:** 7 digits (Numeric, `S9(5)V99`).

Output File 1: New Employee Master File (`NEWEMP`) This file will contain the updated roster of active employees. It must use the exact same 57-byte record layout as the `OLDEMP` file.

Output File 2: Error Transaction File (`ERRTRAN`) This file will hold any HR transactions that cannot be processed due to logical errors. It must use the exact same 50-byte record layout as the `EMPTRAN` file.

2. Processing Specifications

You must use the standard balanced-line algorithm to process the files, comparing the Master Employee ID to the Transaction Employee ID to determine the correct action.

Condition A: Master ID < Transaction ID

- The old master record has no pending HR transactions.
- Write the old master record directly to the `NEWEMP` file without making any changes.

Condition B: Master ID = Transaction ID Evaluate the Action Code to determine the next step:

- **If Code `D` (Delete/Terminate):** The employee is leaving the company. Do not write the record to the new master file. Simply read the next master and transaction records.
- **If Code `C` (Change):** Update the employee's master record. Only move the transaction data to the master record if the transaction field is not blank (for alphanumeric fields) or not zero (for numeric fields). *Note: Accrued hours are never updated via this transaction file.*
- **If Code `A` (Add/Hire):** This is a logical error (Matched Addition). You cannot hire an employee ID that already exists in the system. Write the transaction record to the error file (`ERRTRAN`).

Condition C: Master ID > Transaction ID Evaluate the Action Code to determine the next step:

- **If Code `A` (Add/Hire):** Format a new master record using the data from the transaction record. You must explicitly move zeros to the "Accrued Vacation Hours" and "Accrued Sick Hours" fields, as new hires start with zero balances. Write the formatted record to the new master file (`NEWEMP`).
- **If Code `D` or `C` (Delete or Change):** This is a logical error (Unmatched Delete or Unmatched Change). You cannot terminate or update an employee that does not exist in the master file. Write the transaction record to the error file (`ERRTRAN`).

3. Error Handling & Validation

Your program must define a `FILE STATUS` variable for both output files (`NEWEMP` and `ERRTRAN`).

- After every `WRITE` statement to these files, check the status code.
- If the write is unsuccessful (status code is not `00`), display an error message to the system console indicating the file name, the Employee ID being processed, and the specific File Status code.
- If a write error occurs, gracefully force the program to terminate by setting your primary processing switch to indicate all records have been processed.

1. Old Employee Master File (`OLDEMP.DAT`)

This file contains 4 records. Every line is exactly **57 characters** long, and they are sorted by the 5-character Employee ID.

```
10001JOHN SMITH                    SALESS1045000004001550
10005ALICE JOHNSON                 IT   T3085000012004500
10008ROBERT DAVIS                  HR   H2062000008003000
10012MARY WILLIAMS                 ACCT A1055000005002000
```

2. Personnel Transaction File (`EMPTRAN.DAT`)

This file contains 6 HR transactions. Every line is exactly **50 characters** long, and they are sorted by the 5-character Employee ID.

```
A10003NEW HIRE ONE                  SALESS10400000
D10005                                     0000000
C10008                                   H30650000
C10009GHOST EMPLOYEE                IT   T10500000
A10012MARY WILLIAMS                 ACCT A10550000
D10015                                     0000000
```

Successful Updates (Should appear in `NEWEMP.DAT`):

- **ID 10001:** *Unmatched Master.* John Smith has no pending HR actions. His record should copy directly over to the new file unchanged.
- **ID 10003:** *Good Add (Master > Trans).* This is a new hire. A new 57-byte master record should be created. The student's code must explicitly initialize the vacation (000) and sick hours (00000) at the end of the record before writing it.
- **ID 10005:** *Good Delete (Master = Trans).* Alice Johnson was terminated (`D`). Her record is simply bypassed and should *not* appear in the new master file.
- **ID 10008:** *Good Change (Master = Trans).* Robert Davis received a promotion and a raise (`C`). His name and department were left blank on the transaction, so the program should leave those alone, but update his Job Class to `H3` and Salary to `0650000`.

Logical Errors (Should appear in `ERRTRAN.DAT`):

- **ID 10009:** *Unmatched Change.* The transaction file tries to change the details for a "Ghost Employee", but ID 10009 does not exist in the master file.
- **ID 10012:** *Matched Add.* The transaction file tries to hire Mary Williams, but she is already active in the master file.
- **ID 10015:** *Unmatched Delete.* The transaction file tries to terminate an employee that does not exist in the master file.

Additional requirements:

1) Must use GitHub
    a. Invite me to your repository (dejohns2)
    b. Numerous commits which will prove who did work
    c. Use folders to organize your files in your repository.
    d. Balance work across code and README
        i. Fully explain what the program does
        ii. Embedded screenshots (stored in a subfolder)
        iii. Bullet point list of what new concepts were used
        iv. links to the author's profiles

2) Include either a peer-review or self-review

3) Attach the following files to your assignment:
   a. your program file (SEQ3000.txt) to the assignment as a **TEXT File (.txt)**
   b. along with your Good, Bad, and Ugly (GBU) reflection written at a college level (including name, date, and assignment) **IMPORTANT:** Include, your GitHub URL in this document.